2025년 1학기 데이터통신 14주차
과제 보고서

-Go Back N-

202001156 정보통계학과 김수영

과제 해결

코드 설명:

```python
# 2바이트 1의 보수 합 체크섬 계산
def calculate_checksum(data):
    checksum = 0
    for i in range(0, len(data), 2):
        if i + 1 < len(data):
            word = (data[i] << 8) + data[i+1]
        else:
            word = data[i] << 8
        checksum += word
        checksum = (checksum & 0xFFFF) + (checksum >> 16)
    # 2바이트 1의 보수 합
    return (~checksum) & 0xFFFF

# 수신된 패킷 검증
def verify_packet(packet):
    try:
        if len(packet) < 4:
            return False, -1, b''

        # 조건 만족: 'Seq+Checksum+Data' 전체 패킷에 대한 체크섬을 계산하여 0이 되는지 확인
        if calculate_checksum(packet) != 0:
            # 손상된 패킷이어도 시퀀스 번호는 추출 시도
            try:
                seq_num, _ = struct.unpack('>HH', packet[:4])
            except:
                seq_num = -1
            return False, seq_num, b''

        seq_num, _ = struct.unpack('>HH', packet[:4])
        data = packet[4:]
        return True, seq_num, data

    except (struct.error, TypeError):
        return False, -1, b''


def create_ack_packet(ack_num):
    ack_bytes = struct.pack('>H', ack_num)
    checksum = calculate_checksum(ack_bytes)
    checksum_bytes = struct.pack('>H', checksum)
    return ack_bytes + checksum_bytes

def request_info(sock, server_addr, filename, timeout=5.0):
    try:
        sock.sendto(f'INFO {filename}'.encode('utf-8'), server_addr)
        sock.settimeout(timeout)
        data, addr = sock.recvfrom(1024)

        if addr != server_addr:
            print("Client: Received response from unexpected address")
            return None

        size_str = data.decode('utf-8')
        if size_str.isdigit():
            return int(size_str)
        else:
            print(f"Client: Server error: {size_str}")
            return None

    except socket.timeout:
        print("Client: File info request timeout")
        return None
    except Exception as e:
        print(f"Client: Error requesting file info: {e}")
        return None

```

체크섬 및 info 확인을 요청하는 함수이다. Server, Client 모두 동일하게 구현을 하였다.
struct.pack('!H', ~)를 통해 2바이트 크기의 공간을 생성하므로 ack와 checksum의 바이트 수는 명시적으로 2바이트로 고정시켰으며

```python
def main():
    parser = argparse.ArgumentParser(description="Stop-and-Wait ARQ File Server")
    parser.add_argument('--port', type=int, default=10000, help="Server port")
    parser.add_argument('--dir', type=str, default='.', help="Directory of files to serve")
    parser.add_argument('--mtu', type=int, default=1500, help="Maximum Transmission Unit")
    parser.add_argument('--timeout', type=float, default=0.5, help="Timeout in seconds")
    parser.add_argument('--retries', type=int, default=5, help="Maximum retries")
    args = parser.parse_args()
```

데이터의 크기는 mtu라는 인수를 통해 1500으로 설정하여 데이터의 크기가 최대 1496이 되도록 설정하였다.

```python
                if is_valid:
                    if seq_num == expected_seq: # 조건 만족: seq_num이 예상 시퀀스 번호와 일치
                        if not data: # 전송 종료 신호
                            end_time = time.time()
                            print("\nClient: File transfer complete")

                            total_time = end_time - start_time
                            if total_time > 0 and filesize > 0:
                                throughput_bps = (filesize * 8) / total_time
                                print(f"Client: Total time: {total_time:.2f} seconds")
                                print(f"Client: Throughput: {throughput_bps:,.0f} bps ({throughput_bps/1_000_000:.2f} Mbps)")    "Mbps": Unknown word.
                                if duplicate_count > 0:
                                    print(f"Client: Duplicates received: {duplicate_count}")
                                if corrupted_count > 0:
                                    print(f"Client: Corrupted packets: {corrupted_count}")

                            # 마지막 ACK 전송
                            ack_packet = create_ack_packet(1 - expected_seq)
                            sock.sendto(ack_packet, server_addr)
                            break

                        f.write(data)
                        received_bytes += len(data)
                        print(f"\rClient: [Seq={seq_num}] Received. {received_bytes}/{filesize} bytes ({100*received_bytes/filesize:.1f}%)", end='')

                        ack_packet = create_ack_packet(1 - expected_seq)
                        sock.sendto(ack_packet, server_addr)
                        expected_seq = 1 - expected_seq

                    else:
                        # 조건 만족: case 3. if not ok or seq != expected_seq: 직전 ACK 재전송, 데이터는 버림
                        duplicate_count += 1
                        print(f"\nClient: [Seq={seq_num}] Duplicate packet received. (Expected: {expected_seq})")
                        # 해당 패킷에 대한 ACK 재전송 (seq_num에 대한 ACK는 1-seq_num)
                        ack_packet = create_ack_packet(1 - seq_num)
                        sock.sendto(ack_packet, server_addr)
                        # 데이터는 버림 (write하지 않음)
                else:
                    # 조건 만족: case 3. 손상된 패킷은 무시하고 타임아웃 대기
                    corrupted_count += 1
                    print(f"\nClient: [Seq={seq_num}] Corrupted packet received. Ignored.")
                    # 손상된 패킷은 무시하고 타임아웃을 기다림 (ACK 보내지 않음)

            except socket.timeout:
                print("\nClient: Server response timeout. Exiting.")
                return

        # 조건 만족: timeline의 2배 대기, 늦게온 중복데이터 파악(Seq값) -> ACK값 다시 보내주기
        sock.settimeout(timeout * 2) # timeline의 2배 대기
        final_ack_count = 0
        try:
            while final_ack_count < 3:  # 최대 3번의 추가 ACK 전송
                packet, addr = sock.recvfrom(2048)
                if addr == server_addr:
                    is_valid, seq_num, data = verify_packet(packet)
                    if is_valid and not data: # 서버가 마지막 패킷을 재전송한 경우
                        print("Client: Server's last packet retransmission detected. Resending ACK")
                        # 늦게온 중복데이터 파악 후 ACK 다시 보내기
                        ack_packet = create_ack_packet(1 - seq_num)
                        sock.sendto(ack_packet, server_addr)
                        final_ack_count += 1
        except socket.timeout:
            print("Client: Final confirmation complete. Closing client.")
```

Stop and Wait의 클라이언트 코드이다. 케이스에 대하여 설명을 해보자면

직전에 받은 seq가 예상과 다를 경우 직전 ACK를 재전송하고 데이터는 버리는 식으로 구현을 하였으며 손상된 패킷의 경우 재전송을 기다리는 방식으로 구현을 하였다. 또한, 마지막에 timeline의 2배를 대기하고

늦게 온 중복 데이터를 파악하여 ACK값을 다시 보내는 방식으로 구현을 하였다. 이러한 과정으로 마지막 데이터까지 받으면 클라이언트는 종료된다.

```python
# 서버 조건 1,2: 타임아웃 시 재전송 & 마지막 데이터 재전송
def send_chunk_saw(sock, client_addr, data, seq_num, timeout=0.5, max_retries=10):
    packet = create_packet(seq_num, data)
    expected_ack = 1 - seq_num

    # ========== 서버 조건 1: Server -> client 통신 시 Time expired: Server가 재전송 ==========
    # 최대 재전송 횟수까지 반복
    for attempt in range(max_retries):
        try:
            sock.sendto(packet, client_addr)
            print(f"Server: [Seq={seq_num}] Sending data (Attempt {attempt+1}/{max_retries})")

            # ACK 대기 (타임아웃 설정)
            sock.settimeout(timeout)
            ack_packet, addr = sock.recvfrom(1024)

            if addr == client_addr:
                is_valid, ack_num = verify_ack(ack_packet, expected_ack)
                if is_valid:
                    print(f"Server: [Ack={ack_num}] Received successfully")
                    return True # 성공적으로 ACK 수신
                else:
                    print(f"Server: Invalid ACK received (Expected: {expected_ack}, Got: {ack_num})")

        except socket.timeout:
            # 조건 만족: Time expired 시 Server가 재전송
            print(f"Server: [Seq={seq_num}] Timeout occurred (Attempt {attempt+1})")
            continue # 재전송 (다음 반복으로)
        except Exception as e:
            print(f"Server: Error during transmission: {e}")
            continue

    print(f"Server: [Seq={seq_num}] Failed after {max_retries} attempts")
    return False # 전송 실패
```

서버에 대한 코드이다. ACK를 대기하여 성공적으로 ACK를 받으면 ACK를 업데이트하고 다음 패킷을 보낸다. 그렇지 않고 time expired시 재전송을 한다.

```python
# 파일 다운로드 처리
def handle_download(sock, client_addr, filename, file_table, mtu):
    info = file_table.get(filename)
    if not info:
        try:
            sock.sendto(b'404 Not Found', client_addr)
        except Exception as e:
            print(f"Server: Error sending 404 response: {e}")
        return

    print(f"Server: [{client_addr[0]}:{client_addr[1]}] File download request: {filename}")

    try:
        with open(info['path'], 'rb') as f:
            seq_num = 0 # 0부터 시작
            data_size = mtu - 4
            total_chunks = 0
            successful_chunks = 0

            while True:
                chunk = f.read(data_size)
                if not chunk:
                    break

                total_chunks += 1
                if send_chunk_saw(sock, client_addr, chunk, seq_num):
                    successful_chunks += 1
                    # 조건 만족: 시퀀스 번호 토글 (0 <-> 1) - 0-1-0-1 순서
                    seq_num = 1 - seq_num
                else:
                    print(f"Server: Chunk transmission failed, aborting file transfer")
                    return

            # 마지막 빈 패킷 전송 (종료 신호)
            # ========== 서버 조건 2: 마지막 데이터 전송 시 Time Expired 처리 ==========
            # 조건 만족: 마지막 Data를 보내고 나서도 대기; 마지막 ACK을 받을 때까지 대기 with ARQ
            if send_chunk_saw(sock, client_addr, b'', seq_num):
                # 마지막 빈 패킷도 send_chunk_saw()를 통해 동일한 ARQ 적용
                # 타임아웃 시 재전송
                # 마지막 ACK를 받을 때까지 대기 (최대 max_retries까지)
                print(f"Server: {filename} sent successfully ({successful_chunks}/{total_chunks} chunks)")
            else:
                print(f"Server: Failed to send end-of-file signal")
```

위에서 정의한 함수를 호출하며 마지막 패킷은 빈 패킷을 보낸다. 이를 종료 신호로 사용한다. 이후 마지막 ACK를 받을 때까지 대기를 하며 타임아웃이 발생할 경우 다시 한번 종료 신호를 보낸다.

```python
def next_seq(seq_num):
    """다음 시퀀스 번호 계산 (0~15 순환)"""
    return (seq_num + 1) % 16
```

GBN의 클라이언트, 서버 코드에서 다음 시퀀스 번호 계산을 다음과 같이 선언하여 명시적으로 시퀀스 번호를 계산할 수 있게 하였다.

```python
def is_seq_duplicate(expected_seq, received_seq):
    """받은 시퀀스가 이미 받은 중복/이전 패킷인지 확인"""
    # expected_seq 이전의 패킷들만 중복으로 판단

    # 순환 거리 계산 (received_seq에서 expected_seq까지의 거리)
    if received_seq < expected_seq:
        backward_distance = expected_seq - received_seq
    else:
        backward_distance = (16 - received_seq) + expected_seq

    # 뒤쪽 거리가 8 이하이고 0보다 크면 중복 (이전 패킷)
    return backward_distance <= 8 and backward_distance > 0
```

순환에 대한 구현 부분이다. Expected_seq 이전의 패킷들은 당연히 중복이 될 것이며 순환 거리를 계산하여 뒤쪽 거리가 8 이하이고 0보다 크면 중복 데이터 즉, 중복 패킷으로 판단할 수 있다.

```python
    try:
        with open(out_path, 'wb') as f:
            while True:
                try:
                    sock.settimeout(timeout)
                    packet, addr = sock.recvfrom(2048)

                    if addr != server_addr:
                        continue

                    is_valid, seq_num, data = verify_packet(packet)

                    if is_valid:
                        if is_seq_in_order(expected_seq, seq_num):
                            if not data: # 전송 종료 신호
                                end_time = time.time()
                                print("\nClient: File transfer complete")

                                total_time = end_time - start_time
                                if total_time > 0 and filesize > 0:
                                    throughput_bps = (filesize * 8) / total_time
                                    print(f"Client: Total time: {total_time:.2f} seconds")
                                    print(f"Client: Throughput: {throughput_bps:,.0f} bps ({throughput_bps/1_000_000:.2f} Mbps)")    "Mbps": Unknown word.
                                    if out_of_order_count > 0:
                                        print(f"Client: Out-of-order packets: {out_of_order_count}")
                                    if corrupted_count > 0:
                                        print(f"Client: Corrupted packets: {corrupted_count}")

                                # 마지막 ACK 전송
                                ack_packet = create_ack_packet(seq_num)
                                sock.sendto(ack_packet, server_addr)
                                break

                            f.write(data)
                            received_bytes += len(data)
                            print(f"\rClient: [Seq={seq_num}] Received. {received_bytes}/{filesize} bytes ({100*received_bytes/filesize:.1f}%)", end='')

                            # 조건 만족: 올바른 순서의 패킷에 대한 ACK 전송
                            ack_packet = create_ack_packet(seq_num)
                            sock.sendto(ack_packet, server_addr)
                            # 조건 만족: 시퀀스 번호 증가 (0~15 순환)
                            expected_seq = next_seq(expected_seq)
                        elif is_seq_duplicate(expected_seq, seq_num):
                            print(f"\nClient: [Seq={seq_num}] Duplicate packet. (Expected: {expected_seq})")
                            last_correct_seq = (expected_seq - 1) % 16
                            ack_packet = create_ack_packet(last_correct_seq)
                            sock.sendto(ack_packet, server_addr)
                        else:
                            # 중복 패킷킷
                            out_of_order_count += 1
                            print(f"\nClient: [Seq={seq_num}] Out-of-order packet. (Expected: {expected_seq})")
                            last_correct_seq = (expected_seq - 1) % 16
                            ack_packet = create_ack_packet(last_correct_seq)
                            sock.sendto(ack_packet, server_addr)
                    else:
                        # 손상된 패킷 수신
                        corrupted_count += 1
                        print(f"\nClient: [Seq={seq_num}] Corrupted packet received. Ignored.")
                        # 손상된 패킷은 무시 (ACK 보내지 않음)

                except socket.timeout:
                    print("\nClient: Server response timeout. Exiting.")
                    return
```

Go Back N의 클라이언트에서 다운로드 함수이다. 순서가 틀린 패킷은 버리고 마지막으로 올바르게 받은 패킷의 ACK를 재전송하는 방식으로 구현을 하였다. 이때, 중복 패킷인지 아닌지 판단을 하여 클라이언트에 중복과 out of order를 판단한 뒤 ACK만 다시 보내는 식으로 구현을 하였다.

```
152         sock.settimeout(timeout * 2)
153         final_ack_count = 0
154         try:
155             while final_ack_count < 3:
156                 packet, addr = sock.recvfrom(2048)
157                 if addr == server_addr:
158                     is_valid, seq_num, data = verify_packet(packet)
159                     if is_valid and not data:
160                         print("Client: Server's last packet retransmission detected. Resending ACK")
161                         ack_packet = create_ack_packet(seq_num)
162                         sock.sendto(ack_packet, server_addr)
163                         final_ack_count += 1
164         except socket.timeout:
165             print("Client: Final confirmation complete. Closing client.")
```

마지막 ACK 역시 두 배만큼의 타임아웃을 기다리며 서버가 마지막 패킷을 보냄을 확인하면 다시 패킷을 보낸다.

```python
    def send_packets(self, data_chunks):
        """Go-Back-N ARQ로 패킷들 전송"""
        chunk_index = 0
        total_chunks = len(data_chunks)

        ack_thread = threading.Thread(target=self._ack_receiver)
        ack_thread.daemon = True
        ack_thread.start()

        while chunk_index < total_chunks or len(self.window) > 0:
            with self.lock:
                while (len(self.window) < self.window_size and
                       chunk_index < total_chunks):

                    seq_num = self.next_seq
                    packet_data = data_chunks[chunk_index]
                    packet = create_packet(seq_num, packet_data)

                    try:
                        self.sock.sendto(packet, self.client_addr)
                        print(f"Server: [Seq={seq_num}] Sent packet {chunk_index+1}/{total_chunks}")

                        self.window[seq_num] = (packet, time.time())

                        if len(self.window) == 1:
                            self._start_timer()

                    except Exception as e:
                        print(f"Server: Error sending packet {seq_num}: {e}")
                        break

                    self.next_seq = next_seq(self.next_seq)
                    chunk_index += 1

        while len(self.window) > 0:
            time.sleep(0.001)

        self._send_end_packet()

        self.running = False

    def _send_end_packet(self):
        """Case 3: 파일 전송 완료 신호 패킷 전송"""
        seq_num = self.next_seq
        packet = create_packet(seq_num, b'')
        with self.lock:
            self.sock.sendto(packet, self.client_addr)
            print(f"Server: [Seq={seq_num}] Sent end-of-file packet")

            self.window[seq_num] = (packet, time.time())
            self.next_seq = next_seq(self.next_seq)

            self._start_timer()

        while len(self.window) > 0:
            time.sleep(0.01)

        print("Server: Final ACK received, transmission complete")
```

GBN 서버의 패킷 전송 코드와 종료 패킷 관련 함수이다. 마찬가지로 클라이언트의 ACK를 받을 때까지 기다린다.

```python
    def _timeout_handler(self):
        with self.lock:
            if len(self.window) > 0:
                # 윈도우 내 모든 패킷 재전송
                print(f"Server: Timeout occurred. Retransmitting window from seq {self.base}")
                for i in range(self.window_size):
                    seq_num = (self.base + i) % 16
                    if seq_num in self.window:
                        packet, _ = self.window[seq_num]
                        try:
                            self.sock.sendto(packet, self.client_addr)
                            print(f"Server: [Seq={seq_num}] Retransmitted")
                            self.window[seq_num] = (packet, time.time())
                        except Exception as e:
                            print(f"Server: Error retransmitting packet {seq_num}: {e}")

                self._restart_timer()
```

타임아웃 및 재전송을 하는 함수이다. 타임아웃이 발생하면 재전송을 하여 Case 1을 처리한다.

결과

```
Server: [Ack=14] Received ACK
Server: [Ack=15] Received ACK
Server: [Ack=0] Received ACK
Server: [Seq=13] Sent packet 846/860
Server: [Seq=14] Sent packet 847/860
Server: [Seq=15] Sent packet 848/860
Server: [Ack=1] Received ACK
Server: [Ack=2] Received ACK
Server: [Ack=3] Received ACK
Server: [Ack=4] Received ACK
Server: [Ack=5] Received ACK
Server: [Ack=6] Received ACK
Server: [Ack=7] Received ACK
Server: [Ack=8] Received ACK
Server: [Ack=9] Received ACK
Server: [Ack=10] Received ACK
Server: [Ack=11] Received ACK
Server: [Ack=12] Received ACK
Server: [Ack=13] Received ACK
Server: [Ack=14] Received ACK
Server: [Ack=15] Received ACK
Server: [Seq=0] Sent packet 849/860
Server: [Seq=1] Sent packet 850/860
Server: [Seq=2] Sent packet 851/860
Server: [Seq=3] Sent packet 852/860
Server: [Seq=4] Sent packet 853/860
Server: [Seq=5] Sent packet 854/860
Server: [Seq=6] Sent packet 855/860
Server: [Seq=7] Sent packet 856/860
Server: [Seq=8] Sent packet 857/860
Server: [Seq=9] Sent packet 858/860
Server: [Seq=10] Sent packet 859/860
Server: [Seq=11] Sent packet 860/860
Server: [Ack=0] Received ACK
Server: [Ack=1] Received ACK
Server: [Ack=2] Received ACK
Server: [Ack=3] Received ACK
Server: [Ack=4] Received ACK
Server: [Ack=5] Received ACK
Server: [Ack=6] Received ACK
Server: [Ack=7] Received ACK
Server: [Ack=8] Received ACK
Server: [Ack=9] Received ACK
Server: [Ack=10] Received ACK
Server: [Ack=11] Received ACK
Server: [Seq=12] Sent end-of-file packet
Server: [Ack=12] Received ACK
Server: Final ACK received, transmission complete
Server: test.jpg sent successfully (860 chunks)
```

```
Client: [Seq=11] Out-of-order packet. (Expected: 5)
Client: [Seq=12] Out-of-order packet. (Expected: 5)
Client: [Seq=14] Duplicate packet. (Expected: 5)
Client: [Seq=15] Duplicate packet. (Expected: 5)
Client: [Seq=0] Duplicate packet. (Expected: 5)
Client: [Seq=1] Duplicate packet. (Expected: 5)
Client: [Seq=2] Duplicate packet. (Expected: 5)
Client: [Seq=3] Duplicate packet. (Expected: 5)
Client: [Seq=4] Received. 988856/1285288 bytes (76.9%))
Client: [Seq=6] Out-of-order packet. (Expected: 5)
Client: [Seq=7] Out-of-order packet. (Expected: 5)
Client: [Seq=8] Out-of-order packet. (Expected: 5)
Client: [Seq=9] Out-of-order packet. (Expected: 5)
Client: [Seq=10] Out-of-order packet. (Expected: 5)
Client: [Seq=11] Out-of-order packet. (Expected: 5)
Client: [Seq=12] Out-of-order packet. (Expected: 5)
Client: [Seq=13] Duplicate packet. (Expected: 5)
Client: [Seq=14] Duplicate packet. (Expected: 5)
Client: [Seq=15] Duplicate packet. (Expected: 5)
Client: [Seq=0] Duplicate packet. (Expected: 5)
Client: [Seq=1] Duplicate packet. (Expected: 5)
Client: [Seq=2] Duplicate packet. (Expected: 5)
Client: [Seq=3] Duplicate packet. (Expected: 5)
Client: [Seq=11] Received. 1285288/1285288 bytes (100.0%)
Client: File transfer complete
Client: Total time: 8.66 seconds
Client: Throughput: 1,187,576 bps (1.19 Mbps)
Client: Out-of-order packets: 108
Client: Final confirmation complete. Closing client.
jellyfish319@jellyfish319-server:/mnt/hgfs/test$
```

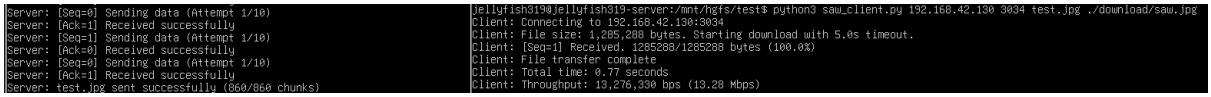
오류가 1% 있을 때의 GBN 결과이다. 중복 감지 및 패킷 재전송이 잘 이루어짐을 확인할 수 있다. Throughput은 1.19Mbps이다.

```
Server: [Ack=0] Received successfully
Server: [Seq=0] Sending data (Attempt 1/10)
Server: [Ack=1] Received successfully
Server: [Seq=1] Sending data (Attempt 1/10)
Server: [Ack=0] Received successfully
Server: [Seq=0] Sending data (Attempt 1/10)
Server: [Ack=1] Received successfully
Server: test.jpg sent successfully (860/860 chunks)
```

```
jellyfish3190@jellyfish319-server:/mnt/hgfs/test$ python3 saw_client.py 192.168.42.130 3034 test.jpg ./download/saw_err.jpg
Client: Connecting to 192.168.42.130:3034
Client: File size: 1,285,288 bytes. Starting download with 5.0s timeout.
Client: [Seq=1] Received. 1285288/1285288 bytes (100.0%)
Client: File transfer complete
Client: Total time: 10.04 seconds
Client: Throughput: 1,024,545 bps (1.02 Mbps)
Client: Final confirmation complete. Closing client.
jellyfish3190@jellyfish319-server:/mnt/hgfs/test$
```

오류가 1% 있을 때 SAW의 결과이다. Throughput은 1.02Mbps이다.

오류가 없을 때의 GBN의 Throughput이다. 15.10Mbps이다.



오류가 없을 때의 SAW의 결과이다. 13.28Mbps이다.

전반적으로 GBM이 더 높은 처리율을 보여줌을 확인할 수 있다.